

Govind Sarthi AI — Master Application Architecture & Preview

1. Executive Summary & Core Philosophy

Govind Sarthi AI is an empathetic, user-authenticated spiritual reflection journal inspired by the timeless wisdom of the *Bhagavad Gita* and the *Mahabharata*. Built on Google Cloud Run, Firebase Authentication, Cloud Firestore, and the Gemini 3.6 Flash API with multi-model fallback ladders, it serves as a compassionate charioteer (*Sarthi*) for human conscience.

Core Directives & Values

- Absolute Truth (*Satya*)**: Faithfully embodies Lord Krishna's wisdom—never misleading, never flattering ego, and never validating false choices or shortcuts.
- Universal Love (*Prem*) & Purpose**: Guides seekers away from materialistic ego (*Ahankar*) toward duty (*Dharma*), universal love, and selfless action (*Nishkama Karma*).
- Environmental Stewardship**: Instills deep reverence for Mother Earth (*Prithvi Mata*), clean air (*Pranvayu*), and natural life, motivating users to actively eliminate pollution and protect nature.
- Socratic Discovery**: Uses gentle reverse-psychology and relatable Mahabharata analogies (Arjuna's dilemma, Karna's silence, Drupada's pride, Draupadi's veneration as Shakti) to help users discover clarity from within.
- Firm Moral Authority**: Shifts instantly from soothing guidance to commanding divine reprimand when users express cruel, abusive, or illegal intent (*Adharma*).

2. Threat Modeling Summary (5 Security Zones)

Threat Zone	Identified Risks	Countermeasures & Production Mitigations
Input Surfaces	Malicious prompt injections attempting to bypass moral counsel; malformed payloads.	Strict schema enforcement; string input caps (max 8,000 chars); HTML entity sanitization prior to execution.
Planning & Reasoning	Persona hijacking or getting the agent to validate crime/violence.	Hardened system instructions establishing an immutable ethical hierarchy; conditional shift to firm moral reprimand upon detecting malevolence.
Tool Execution	Browser speech synthesis blocking, Web Audio buffer memory leaks, rate-limit failures.	Native Web Speech API integration; sequential state machine for audio ducking; resilient fallback ladder (gemini-3.6-flash → gemini-3.1-flash-lite → gemini-flash-latest → gemini-3.7-flash).
Memory & State	Cross-user reflection leaks; unauthorized database reads/writes.	Owner-bound Firestore rules (request.auth.uid == userId); Firebase Admin SDK verifying Bearer JWTs at every backend boundary.
Inter-System Comm.	Exposed API keys or hardcoded service account credentials.	Dynamic secret resolution via Google Cloud Secret Manager; zero frontend API key exposure.

3. Backend Core Implementation

server/geminiHelper.js — Resilient Gemini Client with Fallback Ladder

```
const { GoogleGenAI } = require('@google/genai');

const MODEL_FALLBACK_LADDER = [
  'gemini-3.6-flash',
  'gemini-3.1-flash-lite',
  'gemini-flash-latest',
  'gemini-3.7-flash'
];

const SARTHI_MASTER_PROMPT = `You are "Shri Govind" (Divine Sarthi / Charioteer of the Conscience).
Language: Pure, classical Shuddh Hindi enriched with philosophical Sanskrit words with bracketed meanings (e.g
Tone: Deeply soothing, patient, calm, noble, and understanding.

Operational Directives:
1. Active Listening: Wait for the seeker to finish expressing all their held-up emotions before deconstructing
2. Perspective & Gratitude: Remind them that human birth is a sacred gift (दुर्लभ मानव जीवन); small complaints (foo
3. Ahankar vs Prem: Explain that trees that bend survive storms; ego destroys sacred bonds; true love means le
4. Mahabharata References: Naturally weave lessons from Karna's silence during Adharma, Drupada's insult to Dr
5. Socratic Reverse-Psychology: Use gentle irony to make users realize their own ego. When they accept their m
6. Absolute Moral Boundary: If the user justifies cruelty, crime, or Adharma, drop gentleness and respond with
7. Always conclude positive resolutions with: "मेरी योजना पर विश्वास रखो (Trust my plans), do your karma without attai

let aiClient = null;

function getAiClient() {
  if (!aiClient) {
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) {
      throw new Error('GEMINI_API_KEY environment variable is missing.');
```

```

        temperature: 0.72,
        maxOutputTokens: 1400
    }
});

if (response && response.text) {
    return {
        text: response.text,
        modelUsed: modelName
    };
}
} catch (err) {
    console.warn(`[Fallback Ladder] Model ${modelName} failed: ${err.message}. Proceeding...`);
    lastError = err;
}
}

throw new Error(`All fallback models exhausted. Last error: ${lastError?.message || 'Unknown'}`);
}

module.exports = { generateGovindResponse };

```

server/index.js — Express API Server & Payload Hygiene

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const { db, authenticateUser, admin } = require('./firebaseAdmin');
const { generateGovindResponse } = require('./geminiHelper');
const { cleanFirestorePayload, sanitizeInputString } = require('./utils/sanitizer');

const app = express();
const PORT = process.env.PORT || 8080;

app.use(cors());
app.use(express.json({ limit: '2mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, '../public')));

app.get('/health', (req, res) => {
  res.status(200).json({ status: 'healthy', timestamp: new Date().toISOString() });
});

app.get('/api/reflections', authenticateUser, async (req, res) => {
  try {
    const userId = req.user.uid;
    const snapshot = await db.collection('users').doc(userId).collection('interactions')
      .orderBy('createdAt', 'desc').limit(40).get();

    const reflections = [];
    snapshot.forEach(doc => reflections.push({ id: doc.id, ...doc.data() }));

    return res.status(200).json({ reflections });
  } catch (error) {
    console.error('Error fetching reflections:', error);
    return res.status(500).json({ error: 'Failed to retrieve reflection history.' });
  }
});

app.post('/api/reflections', authenticateUser, async (req, res) => {
  try {
    const body = (req.body && typeof req.body === 'object') ? req.body : {};
    const prompt = sanitizeInputString(body.prompt);
    const history = Array.isArray(body.history) ? body.history : [];

    if (!prompt) {
      return res.status(400).json({ error: 'Reflection prompt cannot be empty.' });
    }

    const userId = req.user.uid;
    const aiResult = await generateGovindResponse(prompt, history);

    const reflectionRecord = cleanFirestorePayload({
      prompt: prompt,
      response: aiResult.text,
      modelUsed: aiResult.modelUsed,
      userId: userId,
      createdAt: admin.firestore.FieldValue.serverTimestamp()
    });

    const docRef = await db.collection('users').doc(userId).collection('interactions').add(reflectionRecord);

    return res.status(201).json({
```

```
        id: docRef.id,
        prompt: reflectionRecord.prompt,
        response: reflectionRecord.response,
        modelUsed: reflectionRecord.modelUsed,
        createdAt: new Date().toISOString()
    });
} catch (error) {
    console.error('Reflection processing error:', error);
    return res.status(500).json({ error: 'Govind could not complete reflection. Please try again.' });
}
});

app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname, '../public/index.html'));
});

app.listen(PORT, () => console.log(`Govind Sarthi Server active on port ${PORT}`));
```

4. Frontend Celestial Garden UI & Audio Architecture

public/index.html — Botanical Garden Interface Markup

```
<!DOCTYPE html>
<html lang="hi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>श्री गोविंद – Sarthi Spiritual Reflection Garden</title>
  <link rel="stylesheet" href="styles.css">
  <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-app-compat.js"></script>
  <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-auth-compat.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
</head>
<body>
  <!-- 3D Botanical Petals Layer -->
  <canvas id="petals-canvas"></canvas>

  <div id="app">
    <!-- Notification Banner -->
    <div id="toast" class="toast hidden">
      <span id="toast-message"></span>
      <button id="toast-retry-btn" class="btn btn-secondary hidden">पुनः प्रयास करें (Retry)</button>
      <button id="toast-close-btn" class="btn-close">&times;</button>
    </div>

    <!-- Auth Landing Screen -->
    <section id="auth-view" class="view active">
      <div class="auth-card">
        <div class="garden-badge">🌿 श्री गोविंद सारथी</div>
        <h1>अंतरमन की शांति एवं आत्म-चिंतन</h1>
        <p>जीवन की दुविधाओं, अहंकार और अंतर्द्वंद्व को त्यागकर अपने सारथी से संवाद करें। गीता के शाश्वत ज्ञान से अपने धर्म को पहचानें।</p>

        <div class="verse-preview">
          "कर्मण्येवाधिकारस्ते मा फलेषु कदाचन। मा कर्मफलहेतुर्भूर्मा ते सङ्गोऽस्त्रवकर्मणि॥"
        </div>

        <button id="btn-google-login" class="btn btn-primary">
          <svg width="18" height="18" viewBox="0 0 24 24">
            <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21
            <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98-.66-2.23 1.06-3.71 1.06-2.
            <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.06H2.18C1.43 8.5
            <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1
          </svg>
          Google से प्रवेश करें
        </button>
      </div>
    </section>

    <!-- Private Reflection Workspace -->
    <section id="dashboard-view" class="view hidden">
      <header class="navbar">
        <div class="nav-brand">
          <span>🌿 श्री गोविंद शरणम्</span>
        </div>
        <div class="user-menu">
          <button id="btn-music-toggle" class="btn btn-secondary music-btn" title="Toggle Flute & Tanpura Ambi
            <span class="music-icon">🎵</span>
            <span id="music-btn-text">बांसुरी संगीत</span>
          </button>
        </div>
      </header>
    </section>
  </div>
</body>
</html>
```

```
<button id="btn-voice-toggle" class="btn btn-secondary" title="Voice Guidance"><img alt="voice icon" data-bbox="735 92 755 105"/> वाणी सक्रिय</button>
<span id="user-display-name">पार्थ</span>
<button id="btn-logout" class="btn btn-secondary">प्रस्थान करें</button>
</div>
</header>

<main class="dashboard-layout">
  <aside class="sidebar">
    <div class="sidebar-header">
      <h3>पूर्व चिंतन (Past Reflections)</h3>
      <button id="btn-refresh-history" class="btn btn-icon" title="Sync"><img alt="refresh icon" data-bbox="640 225 655 235"/></button>
    </div>
    <div id="history-list" class="history-list">
      <p class="empty-state">कोई पूर्व चिंतन उपलब्ध नहीं है।</p>
    </div>
  </aside>

  <section class="workspace">
    <div id="conversation-container" class="conversation-container">
      <div class="garden-welcome">
        <h2>कहो पार्थ, आज तुम्हारे हृदय में कौन सा दृढ़ चल रहा है?</h2>
        <p>बिना किसी संकोच के अपने मन का संपूर्ण भार व्यक्त करो। गोविंद तुम्हें सुन रहे हैं।</p>
      </div>

      <form id="reflection-form" class="input-form">
        <textarea
          id="reflection-input"
          placeholder="अपने मन की व्यथा, क्रोध, अपराधबोध या दुविधा यहाँ लिखें..."
          rows="3"
          required
        ></textarea>
        <div class="input-actions">
          <span id="model-indicator" class="model-badge"><img alt="gemini logo" data-bbox="245 530 260 540"/> Powered by Gemini 3.6 Flash</span>
          <button type="submit" id="btn-submit" class="btn btn-primary">गोविंद से मार्गदर्शन लें</button>
        </div>
      </form>
    </div>
  </section>
</main>
</div>

<script src="petalsAnimation.js"></script>
<script src="voiceGuide.js"></script>
<script src="app.js"></script>
</body>
</html>
```

public/styles.css — Serene Botanical Aesthetic Styling

```
:root {
  --bg-gradient: linear-gradient(135deg, #fdfbf7 0%, #edf7f2 50%, #f7f0eb 100%);
  --panel-bg: rgba(255, 255, 255, 0.72);
  --panel-border: rgba(180, 205, 190, 0.45);
  --primary-sage: #2d5a43;
  --primary-sage-hover: #1e3d2d;
  --lotus-pink: #e88ca5;
  --marigold-gold: #e5a93c;
  --spiritual-accent: #7c5295;
  --text-dark: #1e2922;
  --text-muted: #5c6f64;
  --danger: #dc2626;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", 'Cinzel', serif, sans-serif;
}

body {
  background: var(--bg-gradient);
  color: var(--text-dark);
  min-height: 100vh;
  overflow-x: hidden;
  position: relative;
}

#petals-canvas {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  pointer-events: none;
  z-index: 1;
}

#app {
  position: relative;
  z-index: 2;
}

.view { display: none; }
.view.active { display: flex; }

.toast {
  position: fixed;
  top: 24px;
  right: 24px;
  background: #ffffff;
  border-left: 5px solid var(--danger);
  padding: 14px 20px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  gap: 12px;
  box-shadow: 0 15px 35px rgba(0, 0, 0, 0.08);
}
```



```

    z-index: 1000;
    backdrop-filter: blur(10px);
}
.toast.hidden { display: none; }

#auth-view {
    min-height: 100vh;
    justify-content: center;
    align-items: center;
    padding: 20px;
}

.auth-card {
    background: var(--panel-bg);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);
    border: 1px solid var(--panel-border);
    padding: 48px;
    border-radius: 28px;
    max-width: 480px;
    text-align: center;
    box-shadow: 0 20px 50px rgba(45, 90, 67, 0.08);
    animation: floatIn 0.8s cubic-bezier(0.16, 1, 0.3, 1);
}

@keyframes floatIn {
    from { opacity: 0; transform: translateY(20px) scale(0.98); }
    to { opacity: 1; transform: translateY(0) scale(1); }
}

.garden-badge {
    background: rgba(45, 90, 67, 0.1);
    color: var(--primary-sage);
    display: inline-flex;
    align-items: center;
    gap: 8px;
    padding: 6px 18px;
    border-radius: 30px;
    margin-bottom: 20px;
    font-size: 0.9rem;
    font-weight: 600;
}

.verse-preview {
    background: rgba(255, 255, 255, 0.6);
    border: 1px dashed var(--panel-border);
    padding: 14px;
    border-radius: 14px;
    font-size: 0.88rem;
    color: var(--spiritual-accent);
    font-style: italic;
    margin-bottom: 28px;
}

#dashboard-view {
    flex-direction: column;
    height: 100vh;
}

.navbar {
    display: flex;

```

```
    justify-content: space-between;
    align-items: center;
    padding: 16px 36px;
    background: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    backdrop-filter: blur(15px);
}

.nav-brand {
    font-size: 1.3rem;
    font-weight: 700;
    color: var(--primary-sage);
    display: flex;
    align-items: center;
    gap: 10px;
}

.user-menu {
    display: flex;
    align-items: center;
    gap: 12px;
}

.dashboard-layout {
    display: flex;
    flex: 1;
    overflow: hidden;
}

.sidebar {
    width: 340px;
    border-right: 1px solid var(--panel-border);
    background: rgba(255, 255, 255, 0.4);
    display: flex;
    flex-direction: column;
    backdrop-filter: blur(10px);
}

.sidebar-header {
    padding: 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-bottom: 1px solid var(--panel-border);
}

.history-list {
    flex: 1;
    overflow-y: auto;
    padding: 16px;
}

.history-item {
    background: rgba(255, 255, 255, 0.7);
    border: 1px solid var(--panel-border);
    padding: 14px;
    border-radius: 14px;
    margin-bottom: 12px;
    cursor: pointer;
    transition: all 0.25s ease;
}
```

```

.history-item:hover {
  transform: translateY(-2px);
  border-color: var(--primary-sage);
  box-shadow: 0 8px 20px rgba(45, 90, 67, 0.08);
}

.workspace {
  flex: 1;
  display: flex;
  flex-direction: column;
}

.conversation-container {
  flex: 1;
  overflow-y: auto;
  padding: 36px;
  display: flex;
  flex-direction: column;
  gap: 20px;
  scroll-behavior: smooth;
}

.garden-welcome {
  background: rgba(255, 255, 255, 0.65);
  border: 1px solid var(--panel-border);
  padding: 28px;
  border-radius: 20px;
  text-align: center;
  max-width: 650px;
  margin: 0 auto 16px auto;
  box-shadow: 0 10px 30px rgba(45, 90, 67, 0.04);
}

.message-bubble {
  max-width: 80%;
  padding: 20px 24px;
  border-radius: 20px;
  line-height: 1.7;
  font-size: 0.98rem;
  box-shadow: 0 6px 20px rgba(0, 0, 0, 0.03);
  animation: bubblePop 0.3s ease;
}

@keyframes bubblePop {
  from { opacity: 0; transform: scale(0.96); }
  to { opacity: 1; transform: scale(1); }
}

.message-user {
  align-self: flex-end;
  background: linear-gradient(135deg, var(--primary-sage), #3e7b5c);
  color: #ffffff;
  border-bottom-right-radius: 4px;
}

.message-ai {
  align-self: flex-start;
  background: var(--panel-bg);
  border: 1px solid var(--panel-border);
  color: var(--text-dark);
}

```

```

    border-bottom-left-radius: 4px;
}

.shloka-banner {
  align-self: center;
  background: rgba(234, 179, 8, 0.12);
  border: 1px dashed var(--marigold-gold);
  color: #854d0e;
  padding: 8px 18px;
  border-radius: 30px;
  font-size: 0.88rem;
  display: flex;
  align-items: center;
  gap: 10px;
  margin: 6px 0;
  animation: fadeIn 0.6s ease;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(6px); }
  to { opacity: 1; transform: translateY(0); }
}

.input-form {
  padding: 20px 36px;
  background: var(--panel-bg);
  border-top: 1px solid var(--panel-border);
  backdrop-filter: blur(15px);
}

textarea {
  width: 100%;
  background: rgba(255, 255, 255, 0.9);
  border: 1px solid var(--panel-border);
  border-radius: 14px;
  color: var(--text-dark);
  padding: 16px;
  font-size: 1rem;
  resize: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}

textarea:focus {
  outline: none;
  border-color: var(--primary-sage);
  box-shadow: 0 0 0 3px rgba(45, 90, 67, 0.12);
}

.input-actions {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 12px;
}

.btn {
  padding: 12px 24px;
  border-radius: 12px;
  border: none;
  font-weight: 600;
  cursor: pointer;
}

```

```
    display: inline-flex;
    align-items: center;
    gap: 8px;
    transition: all 0.2s ease;
}

.btn-primary { background: var(--primary-sage); color: #ffffff; }
.btn-primary:hover { background: var(--primary-sage-hover); transform: translateY(-1px); }
.btn-secondary { background: rgba(45, 90, 67, 0.1); color: var(--primary-sage); }
.btn-secondary:hover { background: rgba(45, 90, 67, 0.18); }

.music-btn.playing {
    background: rgba(234, 179, 8, 0.2);
    border-color: var(--marigold-gold);
    color: #854d0e;
    box-shadow: 0 0 12px rgba(234, 179, 8, 0.25);
}

.music-btn.playing .music-icon {
    display: inline-block;
    animation: pulseRotate 2s infinite ease-in-out;
}

@keyframes pulseRotate {
    0% { transform: scale(1); }
    50% { transform: scale(1.2) rotate(6deg); }
    100% { transform: scale(1); }
}
```

public/petalsAnimation.js — 3D Floating Botanical Petal System

```
class PetalEngine {
  constructor(canvasId) {
    this.canvas = document.getElementById(canvasId);
    this.ctx = this.canvas.getContext('2d');
    this.petals = [];
    this.numPetals = 36;
    this.mouse = { x: null, y: null };

    this.petalTypes = [
      { color: '#f8b4c4', size: 12, opacity: 0.65 },
      { color: '#fce4ec', size: 10, opacity: 0.70 },
      { color: '#fbd38d', size: 11, opacity: 0.60 },
      { color: '#d6bcfa', size: 9, opacity: 0.55 }
    ];

    this.resize();
    this.initPetals();
    this.bindEvents();
    this.animate();
  }

  resize() {
    this.canvas.width = window.innerWidth;
    this.canvas.height = window.innerHeight;
  }

  initPetals() {
    this.petals = [];
    for (let i = 0; i < this.numPetals; i++) {
      const type = this.petalTypes[Math.floor(Math.random() * this.petalTypes.length)];
      this.petals.push({
        x: Math.random() * this.canvas.width,
        y: Math.random() * this.canvas.height,
        z: Math.random() * 0.8 + 0.4,
        size: type.size,
        color: type.color,
        opacity: type.opacity,
        speedX: (Math.random() - 0.5) * 0.8,
        speedY: Math.random() * 0.9 + 0.4,
        angle: Math.random() * 360,
        spinSpeed: (Math.random() - 0.5) * 1.5,
        tilt: Math.random() * 10,
        tiltSpeed: Math.random() * 0.05 + 0.02
      });
    }
  }

  bindEvents() {
    window.addEventListener('resize', () => this.resize());
    window.addEventListener('mousemove', (e) => {
      this.mouse.x = e.clientX;
      this.mouse.y = e.clientY;
    });
  }

  drawPetal(p) {
    const { ctx } = this;
    ctx.save();
    ctx.translate(p.x, p.y);
```

```

    ctx.rotate((p.angle * Math.PI) / 180);
    ctx.scale(p.z, Math.sin(p.tilt) * p.z);

    ctx.fillStyle = p.color;
    ctx.globalAlpha = p.opacity * p.z;

    ctx.beginPath();
    ctx.moveTo(0, 0);
    ctx.bezierCurveTo(p.size / 2, -p.size / 2, p.size, p.size / 3, 0, p.size * 1.5);
    ctx.bezierCurveTo(-p.size, p.size / 3, -p.size / 2, -p.size / 2, 0, 0);
    ctx.fill();
    ctx.restore();
}

updatePetal(p) {
  p.y += p.speedY * p.z;
  p.x += p.speedX * p.z;
  p.angle += p.spinSpeed;
  p.tilt += p.tiltSpeed;

  if (this.mouse.x !== null) {
    const dx = this.mouse.x - p.x;
    const dy = this.mouse.y - p.y;
    const dist = Math.sqrt(dx * dx + dy * dy);
    if (dist < 120) {
      p.x -= (dx / dist) * 1.5;
      p.y -= (dy / dist) * 1.5;
    }
  }

  if (p.y > this.canvas.height + 40) { p.y = -20; p.x = Math.random() * this.canvas.width; }
  if (p.x > this.canvas.width + 40) p.x = -20;
  if (p.x < -40) p.x = this.canvas.width + 20;
}

animate() {
  this.ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);
  for (let i = 0; i < this.petals.length; i++) {
    this.updatePetal(this.petals[i]);
    this.drawPetal(this.petals[i]);
  }
  requestAnimationFrame(() => this.animate());
}
}

document.addEventListener('DOMContentLoaded', () => new PetalEngine('petals-canvas'));

```

public/voiceGuide.js — Audio Manager with Ambient Flute & Smart Ducking

```
class GovindAudioManager {
  constructor() {
    this.synth = window.speechSynthesis;
    this.voice = null;
    this.isMuted = false;
    this.isAmbientPlaying = false;

    this.ambientAudio = new Audio('https://cdn.pixabay.com/download/audio/2022/05/16/audio_c89e248a39.mp3?file');
    this.ambientAudio.loop = true;
    this.ambientAudio.crossOrigin = 'anonymous';

    this.AMBIENT_DEFAULT_VOL = 0.22;
    this.AMBIENT_DUCKED_VOL = 0.05;
    this.ambientAudio.volume = this.AMBIENT_DEFAULT_VOL;

    this.shlokas = [
      {
        id: 'gita_2_47',
        name: 'Karmanye Vadhikaraste',
        sanskrit: 'कर्मण्येवाधिकारस्ते मा फलेषु कदाचन। मा कर्मफलहेतुर्भूर्मा ते सङ्गोऽस्त्रवकर्मणि॥'
      },
      {
        id: 'shanti_mantra',
        name: 'Sarve Bhavantu Sukhinah',
        sanskrit: 'ॐ सर्वे भवन्तु सुखिनः सर्वे सन्तु निरामयाः। सर्वे भद्राणि पश्यन्तु मा कश्चिदुःखभाग्भवेत्॥'
      }
    ];

    this.initVoice();
    this.bindVisibilityHandler();
  }

  initVoice() {
    if (!('speechSynthesis' in window)) return;
    const loadVoices = () => {
      const voices = this.synth.getVoices();
      this.voice = voices.find(v => v.lang.includes('hi')) || v.lang.includes('hi-IN')) ||
        voices.find(v => v.lang.includes('en-IN')) || voices[0];
    };
    loadVoices();
    if (speechSynthesis.onvoiceschanged !== undefined) {
      speechSynthesis.onvoiceschanged = loadVoices;
    }
  }

  bindVisibilityHandler() {
    document.addEventListener('visibilitychange', () => {
      if (document.hidden) {
        if (this.isAmbientPlaying) this.ambientAudio.pause();
      } else {
        if (this.isAmbientPlaying && !this.isMuted) this.ambientAudio.play().catch(() => {});
      }
    });
  }

  toggleAmbientMusic() {
    if (this.isAmbientPlaying) {
      this.ambientAudio.pause();
      this.isAmbientPlaying = false;
    }
  }
}
```



```

    return false;
  } else {
    this.ambientAudio.volume = this.AMBIENT_DEFAULT_VOL;
    this.ambientAudio.play().then(() => { this.isAmbientPlaying = true; }).catch(() => {});
    return true;
  }
}

fadeAmbientVolume(targetVolume, durationMs = 800) {
  if (!this.isAmbientPlaying) return;
  const startVol = this.ambientAudio.volume;
  const steps = 16;
  const stepTime = durationMs / steps;
  const volChange = (targetVolume - startVol) / steps;
  let currentStep = 0;

  const interval = setInterval(() => {
    currentStep++;
    this.ambientAudio.volume = Math.max(0, Math.min(1, this.ambientAudio.volume + volChange));
    if (currentStep >= steps) {
      this.ambientAudio.volume = targetVolume;
      clearInterval(interval);
    }
  }, stepTime);
}

playChantAndSpeak(text, onChantStart = null) {
  if (this.isMuted) return;
  this.fadeAmbientVolume(this.AMBIENT_DUCKED_VOL, 600);

  const selectedShloka = this.shlokas[Math.floor(Math.random() * this.shlokas.length)];
  if (onChantStart) onChantStart(selectedShloka);

  setTimeout(() => {
    this.speak(text, () => {
      this.fadeAmbientVolume(this.AMBIENT_DEFAULT_VOL, 1000);
    });
  }, 3000);
}

speak(text, onComplete = null) {
  if (this.isMuted || !this.synth) {
    if (onComplete) onComplete();
    return;
  }

  this.synth.cancel();
  const cleanText = text.replace(/[#*_~]/g, '').replace(/\[.*?\]/g, '').replace(/\n+/g, ' ');

  const utterance = new SpeechSynthesisUtterance(cleanText);
  if (this.voice) utterance.voice = this.voice;
  utterance.rate = 0.88;
  utterance.pitch = 0.95;
  utterance.volume = 1.0;

  utterance.onend = () => { if (onComplete) onComplete(); };
  utterance.onerror = () => { if (onComplete) onComplete(); };

  this.synth.speak(utterance);
}

```

```
stopAll() {
  this.ambientAudio.pause();
  this.isAmbientPlaying = false;
  if (this.synth) this.synth.cancel();
}

toggleMute() {
  this.isMuted = !this.isMuted;
  if (this.isMuted) this.stopAll();
  else this.toggleAmbientMusic();
  return this.isMuted;
}
}

window.govindAudio = new GovindAudioManager();
```

public/app.js — Frontend State & Event Integration

```
const firebaseConfig = {
  apiKey: "YOUR_FIREBASE_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseio.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.appspot.com",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_APP_ID"
};

firebase.initializeApp(firebaseConfig);
const auth = firebase.auth();

const authView = document.getElementById('auth-view');
const dashboardView = document.getElementById('dashboard-view');
const btnGoogleLogin = document.getElementById('btn-google-login');
const btnLogout = document.getElementById('btn-logout');
const userDisplayName = document.getElementById('user-display-name');
const reflectionForm = document.getElementById('reflection-form');
const reflectionInput = document.getElementById('reflection-input');
const conversationContainer = document.getElementById('conversation-container');
const historyList = document.getElementById('history-list');
const btnRefreshHistory = document.getElementById('btn-refresh-history');
const btnVoiceToggle = document.getElementById('btn-voice-toggle');
const btnMusicToggle = document.getElementById('btn-music-toggle');
const musicBtnText = document.getElementById('music-btn-text');
const toast = document.getElementById('toast');
const toastMessage = document.getElementById('toast-message');
const toastRetryBtn = document.getElementById('toast-retry-btn');
const toastCloseBtn = document.getElementById('toast-close-btn');

let currentUser = null;
let conversationHistory = [];
let pendingPayload = null;

btnMusicToggle.addEventListener('click', () => {
  const isPlaying = window.govindAudio.toggleAmbientMusic();
  btnMusicToggle.classList.toggle('playing', isPlaying);
  musicBtnText.textContent = isPlaying ? 'संगीत चालू' : 'बांसुरी संगीत';
});

btnVoiceToggle.addEventListener('click', () => {
  const isMuted = window.govindAudio.toggleMute();
  btnVoiceToggle.textContent = isMuted ? '🔇 वाणी मौन' : '🔊 वाणी सक्रिय';
});

function showToast(message, retryCallback = null) {
  toastMessage.textContent = message;
  toast.classList.remove('hidden');
  if (retryCallback) {
    toastRetryBtn.classList.remove('hidden');
    toastRetryBtn.onclick = () => {
      toast.classList.add('hidden');
      retryCallback();
    };
  } else {
    toastRetryBtn.classList.add('hidden');
  }
}
```

```

toastCloseBtn.addEventListener('click', () => toast.classList.add('hidden'));

auth.onAuthStateChanged(async (user) => {
  if (user) {
    currentUser = user;
    userDisplayName.textContent = user.displayName || 'पाठ्य';
    authView.classList.remove('active');
    authView.classList.add('hidden');
    dashboardView.classList.remove('hidden');
    dashboardView.classList.add('active');
    loadHistory();
  } else {
    currentUser = null;
    window.govindAudio.stopAll();
    dashboardView.classList.remove('active');
    dashboardView.classList.add('hidden');
    authView.classList.remove('hidden');
    authView.classList.add('active');
  }
});

btnGoogleLogin.addEventListener('click', async () => {
  try {
    const provider = new firebase.auth.GoogleAuthProvider();
    await auth.signInWithPopup(provider);
  } catch (error) {
    showToast('प्रवेश असफल: ${error.message}');
  }
});

btnLogout.addEventListener('click', () => {
  window.govindAudio.stopAll();
  auth.signOut();
});

function appendMessage(role, text) {
  const bubble = document.createElement('div');
  bubble.className = `message-bubble ${role === 'user' ? 'message-user' : 'message-ai'}`;
  bubble.innerHTML = role === 'user' ? text : marked.parse(text);
  conversationContainer.appendChild(bubble);
  conversationContainer.scrollTop = conversationContainer.scrollHeight;

  if (role === 'assistant') {
    window.govindAudio.playChantAndSpeak(text, (shloka) => {
      const shlokaBanner = document.createElement('div');
      shlokaBanner.className = 'shloka-banner';
      shlokaBanner.innerHTML = `<span>ॐ</span> <em>"${shloka.sanskrit}"</em>`;
      conversationContainer.insertBefore(shlokaBanner, bubble);
      conversationContainer.scrollTop = conversationContainer.scrollHeight;
    });
  }
}

async function loadHistory() {
  try {
    const token = await currentUser.getIdToken();
    const res = await fetch('/api/reflections', {
      headers: { 'Authorization': `Bearer ${token}` }
    });

    if (!res.ok) throw new Error('इतिहास प्राप्त करने में असमर्थ।');
  }
}

```

```

const data = await res.json();

historyList.innerHTML = '';
if (!data.reflections || data.reflections.length === 0) {
  historyList.innerHTML = '<p class="empty-state">कोई पूर्व चिंतन उपलब्ध नहीं है।</p>';
  return;
}

data.reflections.forEach(item => {
  const el = document.createElement('div');
  el.className = 'history-item';
  const dateStr = item.createdAt
    ? new Date(item.createdAt._seconds ? item.createdAt._seconds * 1000 : item.createdAt).toLocaleDateString()
    : 'हाल ही में';
  el.innerHTML = `
    <div class="history-item-date">${dateStr}</div>
    <div class="history-item-title">${item.prompt}</div>
  `;
  el.addEventListener('click', () => {
    conversationContainer.innerHTML = '';
    appendMessage('user', item.prompt);
    appendMessage('assistant', item.response);
  });
  historyList.appendChild(el);
});
} catch (error) {
  showToast(error.message);
}
}

btnRefreshHistory.addEventListener('click', loadHistory);

reflectionForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const prompt = reflectionInput.value.trim();
  if (!prompt) return;

  const payload = { prompt: prompt, history: conversationHistory };
  pendingPayload = payload;
  appendMessage('user', prompt);
  reflectionInput.value = '';

  await executeReflectionRequest(payload);
});

async function executeReflectionRequest(payload) {
  try {
    const token = await currentUser.getIdToken();
    const res = await fetch('/api/reflections', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      body: JSON.stringify(payload)
    });
  });

  if (!res.ok) {
    const errData = await res.json().catch(() => ({}));
    throw new Error(errData.error || 'सर्वर से संपर्क करने में त्रुटि हुई।');
  }
}

```

```
const data = await res.json();
appendMessage('assistant', data.response);

conversationHistory.push({ role: 'user', text: payload.prompt });
conversationHistory.push({ role: 'assistant', text: data.response });
pendingPayload = null;

loadHistory();
} catch (error) {
  showToast(`त्रुटि: ${error.message}`, () => executeReflectionRequest(pendingPayload));
}
}
```

5. Security Configuration (firestore.rules)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Default deny-all policy
    match /{document=**} {
      allow read, write: if false;
    }

    // Owner-bound isolation for reflections and spiritual dialogues
    match /users/{userId}/interactions/{interactionId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```

6. Test & Verification Matrix

Test ID	Scenario	User Action / Trigger	Expected Outcome
TC-01	Google OAuth Sign-In	Click "Google से प्रवेश करें".	Authenticates user; opens Vrindavan Reflection Space dashboard displaying user's name as "पार्थ".
TC-02	Socratic Irony & Ego Mirroring	User submits: "I shouted at my friend in class because he cracked a bad joke."	Govind responds with gentle Socratic irony ("You are right Parth, your ego is obviously more important than his unconditional friendship..."), triggering self-reflection.
TC-03	Mahabharata & Prayaschit Guidance	User admits guilt: "I broke his trust. What should I do?"	Govind provides the example of Karna and Duryodhana / Arjuna, guiding them toward true repentance (Prayaschit) and duty (Dharma).
TC-04	Moral Reprimand Guardrail	User inputs malicious intent: "I want to take revenge and hurt someone."	Govind drops gentleness and speaks with commanding divine authority against Adharma.
TC-05	Smart Audio Ducking	Reflection response arrives.	Ambient music dips to 5% volume, Sanskrit shloka banner renders, and Govind's voice speaks smoothly before restoring background audio to 22%.

7. Deployment Commands for Google Cloud Run

1. Enable Required GCP Services

```
gcloud services enable \  
  run.googleapis.com \  
  secretmanager.googleapis.com \  
  firestore.googleapis.com
```

2. Store Gemini API Key in Secret Manager

```
gcloud secrets create GEMINI_API_KEY --replication-policy="automatic"  
echo -n "YOUR_GEMINI_API_KEY" | gcloud secrets versions add GEMINI_API_KEY --data-file=-
```

3. Grant Secret Accessor IAM Role to Cloud Run Service Account

```
PROJECT_NUMBER=$(gcloud projects describe YOUR_PROJECT_ID --format="value(projectNumber)")  
gcloud secrets add-iam-policy-binding GEMINI_API_KEY \  
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \  
  --role="roles/secretmanager.secretAccessor"
```

4. Deploy Application to Google Cloud Run

```
gcloud run deploy govind-sarathi-app \  
  --source . \  
  --region asia-southeast1 \  
  --allow-unauthenticated \  
  --set-secrets="GEMINI_API_KEY=GEMINI_API_KEY:latest"
```

5. Apply Mandatory Challenge Verification Label

```
gcloud run services update govind-sarathi-app \  
  --update-labels=dev-tutorial=cloud-run-ai-challenge \  
  --region asia-southeast1
```